

Advanced Java with Web Applications

Complete Topic-Wise Study Guide

BCAC601 | MAKAUT Semester 6

This guide explains every topic from your Advanced Java syllabus in simple, easy-to-understand language with real-world analogies, diagrams, and small code examples — covering Java EE, JDBC, JSP, Servlets, MVC, and Hibernate.

Unit 1	Introduction to Java EE	5 hrs
Unit 2	JDBC	12 hrs
Unit 3	JavaServer Pages (JSP)	15 hrs
Unit 4	Servlets	15 hrs
Unit 5	JSP and Servlet (MVC)	12 hrs
Unit 6	Hibernate Framework	—

Prepared for exam revision • Simple language • Diagrams & code snippets included

Table of Contents

UNIT 1 — Introduction to Java EE

- Overview of Java EE Architecture
- Java SE vs Java EE
- Role of JDBC, JSP, and Servlets in Web Applications

UNIT 2 — JDBC (Java Database Connectivity)

- Introduction to JDBC
- JDBC Drivers and Architecture
- Connecting to Databases
- Executing SQL Queries (SELECT, INSERT, UPDATE, DELETE)
- Statement vs PreparedStatement
- ResultSet and Metadata
- Transaction Management
- Batch Processing
- Commit, Rollback, Save-Point

UNIT 3 — JavaServer Pages (JSP)

- Introduction to JSP
- JSP Lifecycle
- JSP Syntax and Directives
- Scriptlet, Expression and Declaration
- JSP Implicit Objects
- JSP Directives
- JSP Action Elements
- JavaBeans in JSP
- JSP Expression Language (EL)
- JSTL Core Tags
- JSTL Functions
- Custom Tag Library

UNIT 4 — Servlets

- Introduction to Servlets
- Servlet Life Cycle
- Handling HTTP Request and Response
- Servlet Deployment Descriptor (web.xml)
- ServletContext and ServletConfig
- Session Management and Cookies
- Servlet Chaining and Filters
- File Upload and Download in Servlet

UNIT 5 — JSP and Servlet Together

- Combining JSP and Servlets
- Model-View-Controller (MVC) Architecture
- Forwarding Requests between JSP and Servlet

UNIT 6 — Hibernate Framework

- Overview of Hibernate Framework
- Advantages of ORM over JDBC

- Hibernate Architecture and Core Components
- Setting Up Hibernate in a Java Application
- Mapping Java Classes to Database Tables
- Hibernate Configuration (XML and Annotations)
- Basic CRUD Operations
- Hibernate Query Language (HQL) Basics

1.1 Overview of Java EE Architecture

Java EE (Java Enterprise Edition) is a bigger, more powerful version of normal Java, built specially for creating large business applications such as banking systems, online shopping websites, and college portals. While normal Java (Java SE) gives you basic building blocks like loops, classes, and collections, Java EE adds extra tools on top of it that are needed for web applications — like talking to databases, creating dynamic web pages, and managing many users at the same time.

Java EE follows a **multi-tier (layered) architecture**. This means the application is divided into separate layers, and each layer has one clear job. This makes the application easier to build, fix, and upgrade, because changing one layer does not break the others.

Real-life Analogy:

Think of a restaurant. The customer sitting at the table is the **Client Tier** (the browser). The waiter who takes the order and brings food is the **Web/Presentation Tier** (Servlets, JSP). The chef who actually cooks the food using a recipe is the **Business Tier** (your Java logic). And the store room where raw ingredients are kept is the **Database Tier**. Each person only does their own job, and together the restaurant runs smoothly — this is exactly how Java EE layers work together.

Java EE 4-Tier Architecture



1.2 Difference between Java SE and Java EE

Java SE (Standard Edition) is the core Java platform that every Java programmer starts with. It gives you the basics — variables, loops, classes, objects, collections, file handling, and so on. You can build desktop applications and simple console programs with just Java SE.

Java EE (Enterprise Edition) is built *on top of* Java SE. It does not replace Java SE; instead it adds extra libraries and specifications (APIs) specifically needed for large, multi-user, web-based, and networked applications. Things like Servlets, JSP, JDBC, and EJB are all part of Java EE, not Java SE.

Feature	Java SE	Java EE
Purpose	Core/basic programming	Web & enterprise applications
Used for	Desktop apps, console apps	Websites, banking systems, ERPs
Key APIs	Collections, I/O, Threads	Servlets, JSP, JDBC, EJB
Runs on	JVM only	JVM + Application/Web Server
Example	A calculator app	An online food ordering website

1.3 Role of JDBC, JSP, and Servlets in Web Applications

These three technologies are the heart of Java web development, and each plays a different role when a user interacts with a website.

a) JDBC (Java Database Connectivity)

JDBC is the bridge between your Java program and a database like MySQL or Oracle. Whenever your application needs to save, read, update, or delete data — for example storing a new student's marks — JDBC is the technology that actually talks to the database and runs the SQL queries.

b) JSP (JavaServer Pages)

JSP is used to design the **visual part** of a web page — the part the user actually sees, like HTML, but with the power to insert Java code inside it to show dynamic content (for example, displaying the logged-in user's name).

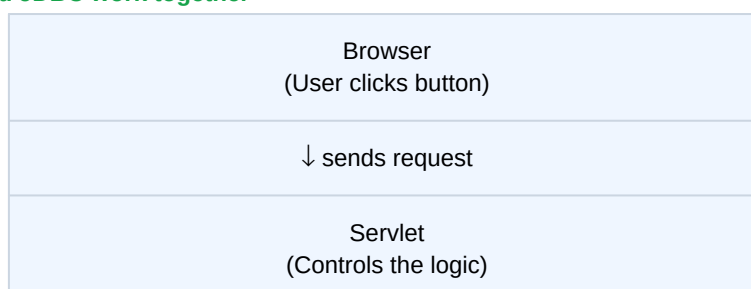
c) Servlets

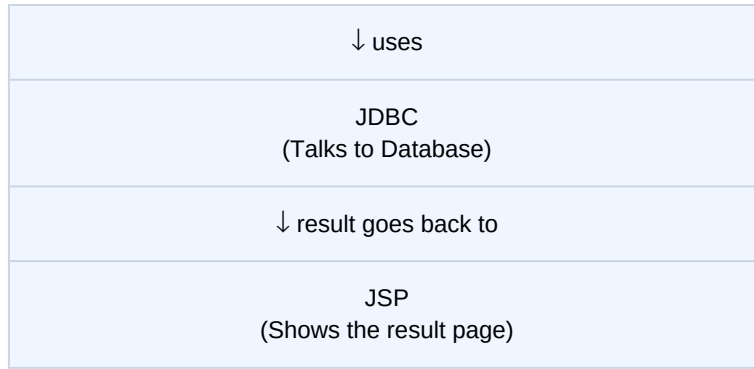
A Servlet is a Java program that runs on the server and handles requests coming from the browser. It receives the request, decides what needs to be done (like checking a password), talks to JDBC if database work is needed, and then sends the response back, often by forwarding to a JSP page to display the result.

Real-life Analogy:

*When you order food on an app: the menu screen you see is the **JSP**. When you tap "Place Order", the **Servlet** receives your request and processes it (checks stock, calculates price). The Servlet then asks **JDBC** to save your order details into the restaurant's database. Finally, the Servlet sends you back to a JSP page that shows "Order Placed Successfully".*

How JSP, Servlet, and JDBC work together





JDBC (Java Database Connectivity)

12 Hours

2.1 Introduction to JDBC

JDBC stands for **Java Database Connectivity**. It is a set of Java classes and interfaces that allow a Java program to connect to a database, send SQL commands, and get back results. Without JDBC, Java would have no standard way of talking to databases like MySQL, Oracle, or PostgreSQL.

JDBC acts like a **translator**. Your Java code does not understand a database's native language, and the database does not understand Java. JDBC sits in between and converts your Java method calls into a form the database understands, and then converts the database's response back into Java objects you can use.

Real-life Analogy:

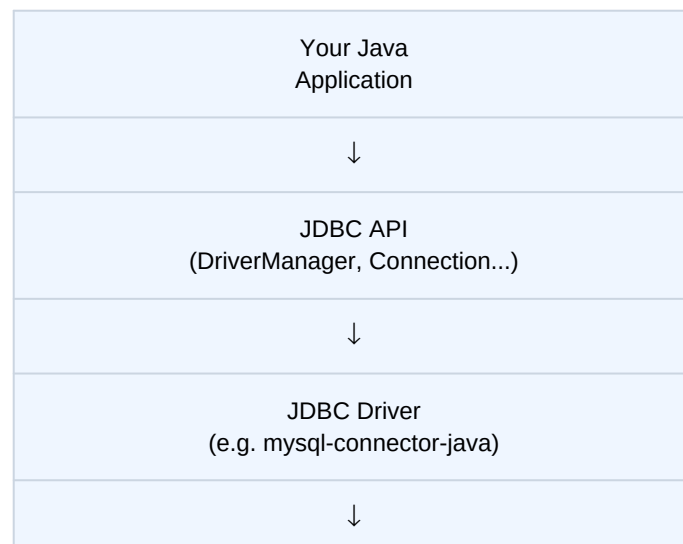
Imagine you only speak English and you want to order food from a chef who only speaks French. You need a translator standing between you. JDBC is that translator standing between your Java program and the database.

2.2 JDBC Drivers and Architecture

A JDBC Driver is the actual piece of software that knows how to communicate with one specific type of database. Since every database (MySQL, Oracle, SQL Server) works slightly differently internally, each database vendor provides its own driver that plugs into the standard JDBC API.

JDBC architecture has two main layers: the **JDBC API** (the standard set of classes your Java code uses, like `Connection`, `Statement`) and the **JDBC Driver** (the vendor-specific code that actually talks to that particular database). There are four types of drivers, but in modern development we almost always use **Type 4 (Thin Driver)**, which is written purely in Java and talks directly to the database over the network — it needs no extra software installed.

JDBC Architecture Flow



Database
(MySQL/Oracle)

2.3 Connecting to Databases

Connecting to a database in JDBC always follows the same basic steps: load the driver, create a connection using a special address called a **JDBC URL** (which contains the database type, server location, and database name), and then use that connection to talk to the database. The connection is like a telephone line that stays open between your program and the database until you close it.

Code Example:

```
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnect {
    public static void main(String[] args) {
        try {
            String url = "jdbc:mysql://localhost:3306/college_db";
            String user = "root";
            String password = "1234";

            Connection con = DriverManager.getConnection(url, user, password);
            System.out.println("Connected successfully!");
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2.4 Executing SQL Queries (SELECT, INSERT, UPDATE, DELETE)

Once connected, JDBC lets you run any SQL query from Java. **SELECT** queries (which return data) are run using `executeQuery()`, which gives back a `ResultSet`. Queries that change data — **INSERT**, **UPDATE**, and **DELETE** — are run using `executeUpdate()`, which returns the number of rows affected instead of data.

Code Example:


```

Statement stmt = con.createStatement();

// INSERT
stmt.executeUpdate(
    "INSERT INTO students VALUES (1, 'Indra', 'CSE')");

// SELECT
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
while (rs.next()) {
    System.out.println(rs.getInt(1) + " " + rs.getString(2));
}

// UPDATE
stmt.executeUpdate(
    "UPDATE students SET branch='IT' WHERE id=1");

// DELETE
stmt.executeUpdate("DELETE FROM students WHERE id=1");

```

2.5 Use of Statement and PreparedStatement

Statement is used to run plain SQL queries, but it has a problem — if you build the query by joining strings (especially with user input), it becomes slow when repeated and unsafe against SQL Injection attacks. **PreparedStatement** solves both problems: the SQL query is written once with placeholders (?) and compiled by the database in advance, so it runs faster on repeated use, and user input is automatically treated as safe data rather than executable SQL code.

Real-life Analogy:

Statement is like writing a brand-new letter by hand every single time you want to send the same message to different people. PreparedStatement is like having a printed template letter with blank spaces, where you just fill in the name each time — much faster and safer.

Code Example (PreparedStatement):

```

String sql = "INSERT INTO students VALUES (?, ?, ?)";
PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 2);
ps.setString(2, "Riya");
ps.setString(3, "ECE");
ps.executeUpdate();

```

2.6 ResultSet and Metadata

A **ResultSet** is the table of data that comes back after running a SELECT query. It works like a cursor that points to one row at a time; calling `rs.next()` moves the cursor to the next row and returns `true` until there are no more rows.

ResultSetMetaData gives you information *about* the data, rather than the data itself — for example, how many columns the result has, what each column is named, and what data type each column holds. This is very useful when you don't know the table structure in advance, such as when building a generic table-viewer.

Code Example:

```

ResultSet rs = stmt.executeQuery("SELECT * FROM students");
ResultSetMetaData md = rs.getMetaData();

int colCount = md.getColumnCount();
for (int i = 1; i <= colCount; i++) {
    System.out.println("Column: " + md.getColumnName(i));
}

```

2.7 Transaction Management

A transaction is a group of database operations that must all succeed together, or none of them should happen at all. For example, when transferring money between two bank accounts, both "subtract from account A" and "add to account B" must succeed together; if only one happens, money disappears or gets duplicated. JDBC lets you control this using `con.setAutoCommit(false)`, which stops the database from saving each statement immediately, so you can decide when to actually save the changes.

By default, JDBC runs in **auto-commit mode**, meaning every single SQL statement is saved to the database immediately on its own. Turning this off is the first step before you can manually manage a transaction.

2.8 Batch Processing

Batch Processing means grouping many SQL statements together and sending them to the database in one single trip, instead of sending them one by one. This is much faster because network/database communication overhead happens only once for the whole group, not once per statement — very useful when inserting hundreds or thousands of rows.

Code Example:

```

PreparedStatement ps = con.prepareStatement(
    "INSERT INTO students VALUES (?, ?, ?)");

ps.setInt(1, 1); ps.setString(2, "Amit"); ps.setString(3, "CSE");
ps.addBatch();

ps.setInt(1, 2); ps.setString(2, "Sneha"); ps.setString(3, "IT");
ps.addBatch();

ps.executeBatch(); // sends both inserts together

```

2.9 Commit, Rollback, Save-Point

These three commands control how a transaction finishes. **Commit** permanently saves all the changes made in the current transaction to the database — once committed, the change is final. **Rollback** cancels and undoes all changes made since the transaction began, as if they never happened — used when something goes wrong midway. A **Save-Point** is a marked checkpoint inside a long transaction; instead of rolling back everything, you can roll back only up to a specific save-point, keeping the earlier safe changes.

Real-life Analogy:

Think of writing a document in Google Docs. **Commit** is like clicking "Save" permanently. **Rollback** is like pressing Ctrl+Z to undo everything you typed since you opened the document. A **Save-Point** is like a bookmark you place midway, so you can undo back to that bookmark instead of erasing everything.

Code Example:

```
con.setAutoCommit(false);

stmt.executeUpdate("UPDATE accounts SET balance=balance-500 WHERE id=1");
Savepoint sp1 = con.setSavepoint("afterDebit");

stmt.executeUpdate("UPDATE accounts SET balance=balance+500 WHERE id=2");

if (somethingWentWrong) {
    con.rollback(sp1); // undo only till savepoint
} else {
    con.commit(); // save everything permanently
}
```

JavaServer Pages (JSP)

15 Hours

3.1 Introduction to JSP

JSP (JavaServer Pages) is a technology that lets you write HTML pages with small pieces of Java code mixed inside them, so the page can show different content depending on the situation — this is called a **dynamic web page**. A plain HTML page always shows the exact same content to everyone, but a JSP page can greet "Hello Indra" to one user and "Hello Riya" to another, because Java code inside it can fetch the correct name.

Behind the scenes, every JSP page is automatically converted into a Servlet by the server before it runs. So JSP is really just a more convenient way of writing a Servlet that is focused mainly on displaying the output page.

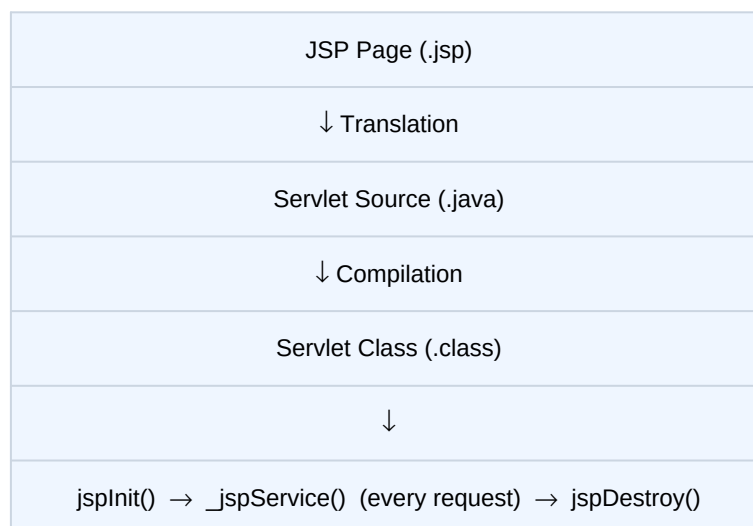
Real-life Analogy:

A JSP page is like a printed greeting card template that has blank spaces for a name. Each time someone uses it, the printing machine (the server) fills in a different name, so the same template produces a personalized card for every person.

3.2 JSP Lifecycle

Just like a Servlet, every JSP page goes through a fixed life cycle managed by the server. The first time a JSP page is requested, the server **translates** it into a Servlet's Java source file, then **compiles** that into a .class file. The server then calls `jspInit()` once to set things up, calls `_jspService()` every single time a request comes in (this is where the actual page content is generated), and finally calls `jspDestroy()` once when the page is being removed from memory.

JSP Life Cycle



3.3 JSP Syntax and Directives

JSP gives you several special tags to mix Java with HTML. **Directives** are special instructions to the JSP engine itself, written as `<%@ ... %>`, that control how the page is processed overall — for example, which language is used, or which other files to include. They don't directly produce visible output; they just configure the page.

3.4 Scriptlet, Expression and Declaration

These are the three ways to write actual Java code inside a JSP page.

a) Scriptlet `<% ... %>`

Used to write any block of plain Java code/logic, such as loops or if-conditions, directly inside the page.

b) Expression `<%= ... %>`

Used to directly print a value onto the page. It is like calling `System.out.print()` but the output goes onto the web page instead of the console. No semicolon is used inside it.

c) Declaration `<%! ... %>`

Used to declare variables or methods that belong to the whole page (like fields of the underlying Servlet class), rather than code that runs every time the page loads.

Code Example:

```
<%! int count = 0; // Declaration - belongs to the class %>

<%
count++; // Scriptlet - Java logic
String name = "Indra";
%>

<p>Visitor name: <%= name %></p> <!-- Expression -->
<p>Page visited <%= count %> times</p>
```

3.5 JSP Implicit Objects

Implicit objects are ready-made objects that the JSP engine automatically creates for every page, so you can use them directly without declaring them yourself. The most commonly used ones are `request` (holds data sent by the browser), `response` (used to send data back), `session` (stores data for one user across multiple pages), `out` (used to write output to the page), and `application` (shared data for the whole web app, across all users).

Object	Purpose
<code>request</code>	Data sent from browser to server
<code>response</code>	Data sent back from server to browser
<code>session</code>	Stores data for one user's visit
<code>application</code>	Shared data for the entire app/all users
<code>out</code>	Writes text/HTML onto the output page

3.6 JSP Directives

There are three main directives. The **page** directive sets properties for the whole page, like the content type or error page. The **include** directive inserts the content of another file into the page at translation time (like copy-pasting a header file into many pages). The **taglib** directive lets you use a custom tag library (such as JSTL) inside the page.

Code Example:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ include file="header.jsp" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

3.7 JSP Action Elements

Action elements are special XML-style tags (written as `<jsp: ... />`) that perform a specific task while the page is running, rather than at translation time like directives. Common ones include `<jsp:include>` (includes another page's output at request time), `<jsp:forward>` (sends the request to another page entirely), and `<jsp:useBean>` (creates or finds a JavaBean object to use on the page).

3.8 Java Beans in JSP

A JavaBean is simply a normal Java class that follows three rules: it has a no-argument constructor, its variables are private, and it has public `getX()` and `setX()` methods to read/write those variables. JSP uses JavaBeans to neatly hold and pass around data (like a Student's name and marks) between pages, using `<jsp:useBean>`, `<jsp:getProperty>`, and `<jsp:setProperty>` tags.

Code Example:

```
public class StudentBean {
    private String name;
    public StudentBean() {}
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Using it on a JSP page:

```
<jsp:useBean id="student" class="StudentBean" />
<jsp:setProperty name="student" property="name" value="Indra" />
<p>Hello, <jsp:getProperty name="student" property="name" /></p>
```

3.9 Introduction to JSP Expression Language (EL)

Expression Language (EL) is a simpler, cleaner way to access data on a JSP page without writing any Java scriptlet code. Instead of writing `<%= student.getName() %>`, you can simply write `${student.name}`. EL can read data stored in request, session, or application scope automatically, and also do simple maths and comparisons, making JSP pages much shorter and easier to read.

Code Example:

```
<!-- Instead of scriptlet -->
<p>Welcome, ${student.name}</p>
<p>Total: ${price * quantity}</p>
```

3.10 Introduction to JSTL Core Tags

JSTL (JSP Standard Tag Library) provides ready-made tags that replace common Java logic like loops and if-conditions, so you almost never need scriptlets in a JSP page. The **Core tag library** covers the most frequently used logic: looping over data, checking conditions, and setting/removing variables.

Tag	Purpose
<c:if>	Runs content only if a condition is true
<c:choose>, <c:when>, <c:otherwise>	Works like if-else-if chains
<c:forEach>	Loops over a list/array, like a for loop
<c:set>	Creates or updates a variable
<c:remove>	Deletes a variable from scope
<c:out>	Safely prints a value onto the page
<c:catch>	Catches exceptions inside the JSP page
<c:url>	Builds a properly formatted URL

Code Example:

```
<c:forEach var="s" items="${studentList}">
<p>${s.name} - ${s.branch}</p>
</c:forEach>

<c:if test="${marks >= 40}">
<p>Result: Pass</p>
</c:if>
```

3.11 JSTL Functions

JSTL also provides built-in functions (used with the `fn` prefix) for common string operations directly inside EL expressions, such as finding the length of text, converting to uppercase/lowercase, or checking if one string contains another — without writing any Java code.

Code Example:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>

<p>${fn:toUpperCase(student.name)}</p>
<p>Name length: ${fn:length(student.name)}</p>
```

3.12 Custom Tag Library

Sometimes the built-in JSTL tags are not enough for a specific need in your project. In that case, you can create your own **Custom Tags** — a Java class (called a Tag Handler) that defines exactly what the tag should do, registered using a `.tld` (Tag Library Descriptor) file. Once created, you can use your custom tag in a JSP page just like any built-in tag, which helps keep repeated logic clean and reusable

across many pages.

4.1 Introduction to Servlets

A Servlet is a Java class that runs inside a web server and handles requests sent by a browser, then sends back a response. Servlets are the foundation of Java web development — they control what should happen when a user submits a form, clicks a link, or visits a particular URL.

Unlike a normal Java program that runs once and stops, a Servlet stays loaded in the server's memory and can handle *many* requests from *many* different users, one after another, without restarting each time. This makes Servlets fast and efficient for web applications.

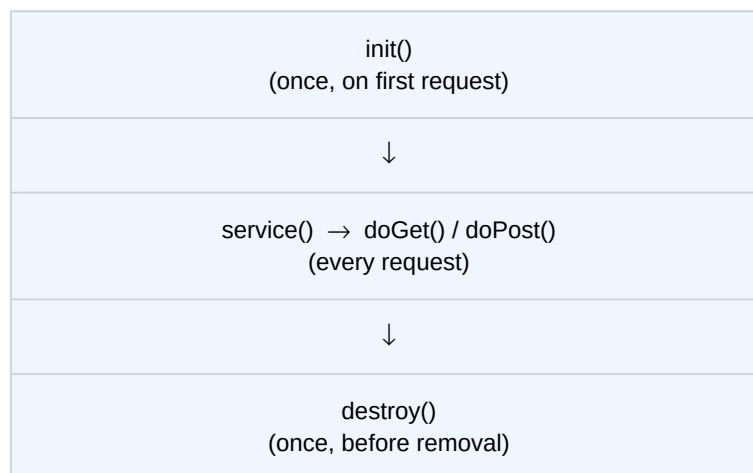
Real-life Analogy:

A Servlet is like a receptionist at an office who stays at the desk all day. Every visitor (request) that comes in is handled by the same receptionist, who decides what to do and replies accordingly — the receptionist doesn't need to be hired fresh for every single visitor.

4.2 Servlet Life Cycle

A Servlet goes through three main stages, all managed automatically by the server. First, `init()` is called only **once**, when the Servlet is loaded for the very first time — used for one-time setup like opening a database connection. Then `service()` is called **every single time** a new request arrives, and internally it calls `doGet()` or `doPost()` depending on the request type. Finally, `destroy()` is called only **once**, right before the Servlet is removed from memory, used for cleanup.

Servlet Life Cycle



4.3 Handling HTTP Request and Responses

Every browser request to a Servlet arrives as an `HttpServletRequest` object, which holds everything the browser sent — form data, headers, the URL, cookies, and more. The Servlet replies using an `HttpServletResponse` object, which is used to send back HTML, set headers, or redirect the browser elsewhere. `doGet()` handles requests where data is visible in the URL (like search queries), while `doPost()` handles requests where data is sent hidden in the request body (like login

forms).

Code Example:

```
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name"); // read input
        res.setContentType("text/html");

        PrintWriter out = res.getWriter();
        out.println("<h2>Hello, " + name + "!</h2>"); // send output
    }
}
```

4.4 Purpose and Use of Servlet Deployment Descriptor File

The Deployment Descriptor is an XML file named `web.xml`, kept inside the `WEB-INF` folder of a web project. Its job is to tell the server important configuration details about the application — mainly, which URL pattern (like `/hello`) should be mapped to which Servlet class, along with other settings like session timeout, welcome pages, and error pages. In modern Java EE, this can also be done using annotations directly in the Servlet code, but understanding `web.xml` is still important.

Code Example:

```
<web-app>
<servlet>
<servlet-name>Hello</servlet-name>
<servlet-class>com.example.HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>Hello</servlet-name>
<url-pattern>/hello</url-pattern>
</servlet-mapping>
</web-app>
```

4.5 ServletContext and ServletConfig

ServletConfig holds configuration information for *one specific* Servlet only — like initialization parameters meant only for that Servlet. **ServletContext**, on the other hand, represents the *entire web application* and is shared by every Servlet and JSP inside it — useful for storing data that all parts of the app need access to, such as a site-wide counter or shared settings.

Real-life Analogy:

ServletConfig is like the personal notes in one employee's diary — only that employee uses it. ServletContext is like the office notice board that every employee in the building can read and use.

4.6 Session Management and Cookie

HTTP is a **stateless** protocol, meaning the server normally forgets everything about a user the moment one request finishes — it does not remember if you were logged in just a second ago. **Session management** solves this by letting the server remember information about a specific user across multiple requests, typically using an `HttpSession` object, identified by a unique session ID. A **Cookie** is a small piece of data that the server sends to the browser, and the browser sends back with every

future request — often used to carry that session ID, or to remember small preferences like a username for "Remember Me".

Code Example:

```
// Storing data in a session
HttpSession session = req.getSession();
session.setAttribute("username", "Indra");

// Reading it back later on another page
String user = (String) session.getAttribute("username");

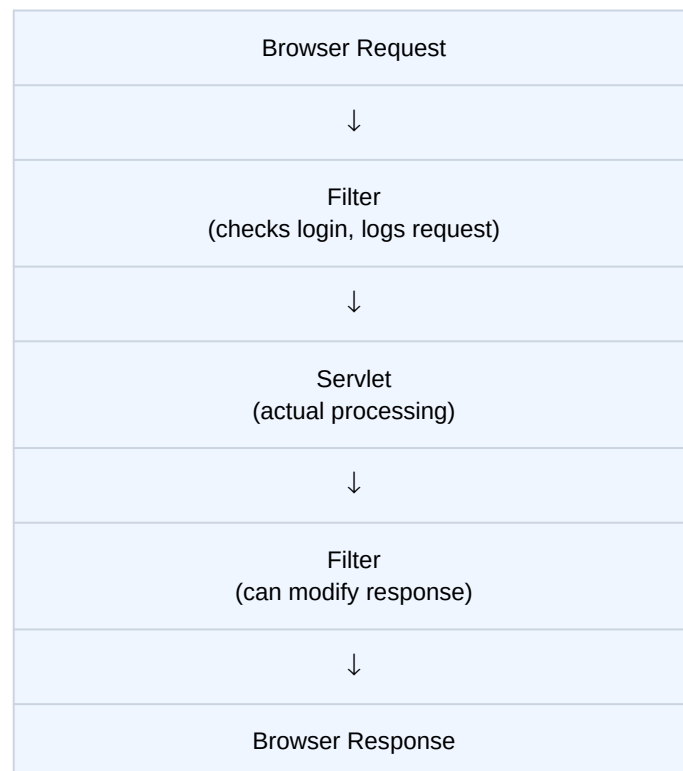
// Creating a cookie
Cookie c = new Cookie("lastVisit", "2026-06-23");
res.addCookie(c);
```

4.7 Servlet Chaining and Filters

Servlet Chaining means the output of one Servlet becomes the input for the next Servlet, so several Servlets process the same request one after another in a chain.

A **Filter** is a special component that can intercept a request *before* it reaches the Servlet, and/or intercept the response *before* it goes back to the browser — without the Servlet even knowing the Filter exists. Filters are commonly used for tasks that apply to many pages at once, like checking if a user is logged in, logging every request, or compressing responses.

How a Filter wraps around a Servlet



4.8 File Upload and Download in Servlet

To allow users to **upload** a file, the HTML form must use `enctype="multipart/form-data"`, and the Servlet reads the uploaded file using the `Part` interface (from the `javax.servlet.http` package), which lets you save the uploaded file's content onto the server's disk.

To allow **download**, the Servlet reads a file from the server's storage and writes its bytes directly into the response's output stream, after setting the correct Content-Type and Content-Disposition headers so the browser knows to prompt a "Save File" dialog instead of trying to display it.

Code Example (Upload):

```
@MultiPartConfig
public class UploadServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        Part filePart = req.getPart("myFile");
        filePart.write("/uploads/" + filePart.getSubmittedFileName());
    }
}
```

5.1 Combining JSP and Servlets

In real projects, JSP and Servlets are almost always used together, because each is good at a different job. Servlets are good at **handling logic** — reading form data, talking to the database, deciding what to do — but writing large amounts of HTML inside a Servlet is messy and hard to maintain. JSP, on the other hand, is good at **displaying output** using HTML-like syntax, but mixing too much Java logic into a JSP page makes it messy too. So the standard approach is: the Servlet does the thinking, and the JSP does the showing.

Real-life Analogy:

It's like a restaurant kitchen and the dining table being separate. The kitchen (Servlet) prepares and decides what food to make. The dining table setup (JSP) is only concerned with how the food looks when served to the customer. Mixing cooking and serving into one place would create chaos.

5.2 Model-View-Controller Architecture

MVC is a design pattern that organizes a web application into three separate parts, each with one clear responsibility, so the application is easier to manage and update.

Model

Represents the application's data and business logic — usually plain Java classes (like JavaBeans) and the code that talks to the database (often using JDBC). The Model doesn't know anything about how data will be displayed.

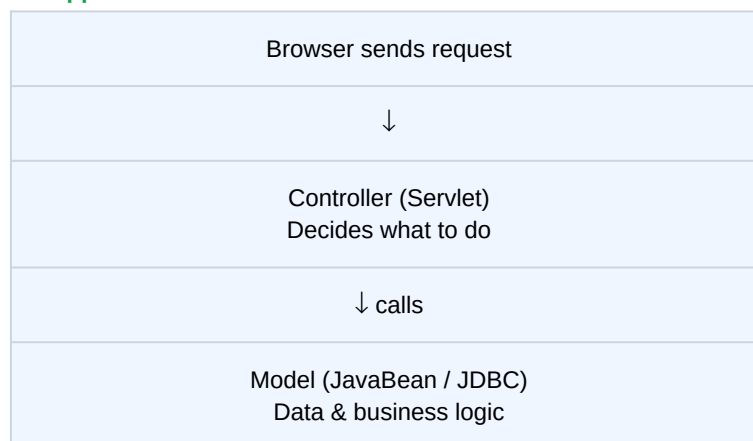
View

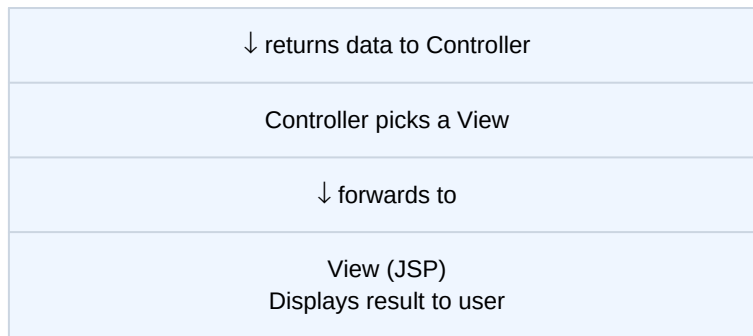
Represents what the user actually sees — this is the JSP page. The View only displays data given to it; it does not contain business logic.

Controller

Represents the decision-maker — this is the Servlet. It receives the user's request, calls the Model to get/update data, and then decides which View (JSP) should be shown next.

MVC Flow in a Java Web Application





MVC's biggest benefit is **separation of concerns**: a designer can change how the View looks without touching the logic, and a developer can change the business logic without breaking the page design. This makes large team projects far easier to manage.

5.3 Forwarding Requests between JSP and Servlet

Forwarding means passing the current request from one resource (a Servlet) to another resource (a JSP page) *on the server itself*, without the browser knowing or making a fresh request. This is done using a `RequestDispatcher`, and it is the standard way a Controller Servlet hands control over to a View JSP in the MVC pattern. Because it happens entirely on the server, the original request data (set using `request.setAttribute()`) is still available on the forwarded page.

Code Example:

```
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {

    String name = req.getParameter("name");

    // Model logic (could call JDBC here)
    req.setAttribute("studentName", name);

    // Controller forwards to the View
    RequestDispatcher rd = req.getRequestDispatcher("result.jsp");
    rd.forward(req, res);
}
```

On `result.jsp`, the data is simply read using EL:

```
<p>Welcome, ${studentName}!</p>
```

Overview of Hibernate Framework

6.1 Overview of Hibernate Framework

Hibernate is a Java framework that handles the work of saving and reading Java objects from a database, without you needing to write SQL queries by hand for every operation. It is an implementation of a concept called **ORM (Object-Relational Mapping)**, which means it automatically converts a Java object (like a Student object) into a database row, and converts a database row back into a Java object — handling the translation between the "object world" of Java and the "table world" of databases.

Real-life Analogy:

Imagine Java objects speak "Object language" and the database speaks "Table language". Hibernate is like a skilled interpreter who automatically converts everything you say in Object language into Table language for the database, and converts the database's replies back into Object language for you — so you never have to learn Table language (SQL) yourself for routine work.

6.2 Advantages of ORM over JDBC

With plain JDBC, you must manually write SQL for every single operation, manually map each column to a Java variable, and manually handle a lot of repetitive boilerplate code. Hibernate (ORM) removes most of this manual work: you simply work with Java objects, and Hibernate generates and runs the correct SQL behind the scenes.

Aspect	JDBC	Hibernate (ORM)
SQL writing	Manual, for every query	Mostly automatic
Database portability	SQL often database-specific	Same code works across DBs
Boilerplate code	High (lots of repetition)	Low (Hibernate handles it)
Caching	Not built-in	Built-in caching support
Object mapping	Manual, row-by-row	Automatic, object-to-table

6.3 Hibernate Architecture and Core Components

Hibernate's architecture has a few key components that work together. The **Configuration** object reads settings (database URL, username, dialect) usually from a file called `hibernate.cfg.xml`. This is used to build a **SessionFactory** — a heavyweight object created only once per application, which knows how to create database connections efficiently. From the SessionFactory, you get a **Session** — a lightweight object representing one single conversation with the database, used to actually save,

update, delete, or fetch objects. Operations are usually wrapped in a **Transaction**, ensuring all the changes succeed or fail together.

Hibernate Core Components Flow



6.4 Setting Up Hibernate in a Java Application

To set up Hibernate, you need three things: the Hibernate library (added via Maven or downloaded jars), a configuration file telling Hibernate how to connect to your database, and your Java entity classes mapped to database tables. Once these are in place, your application can build a `SessionFactory` once at startup, and reuse it throughout the application's lifetime to open `Sessions` whenever needed.

Code Example (basic setup):

```
Configuration cfg = new Configuration();
cfg.configure("hibernate.cfg.xml");

SessionFactory factory = cfg.buildSessionFactory();
Session session = factory.openSession();
```

6.5 Mapping Java Classes to Database Tables

Mapping means telling Hibernate exactly which Java class corresponds to which database table, and which variable corresponds to which column. This can be done using an XML mapping file (the older way) or, more commonly today, using simple annotations directly on the Java class, such as `@Entity` and `@Table`.

Code Example:


```

@Entity
@Table(name = "students")
public class Student {

    @Id
    @Column(name = "id")
    private int id;

    @Column(name = "name")
    private String name;

    // getters and setters
}

```

6.6 Hibernate Configuration (XML and Annotations)

Hibernate can be configured in two ways. The **XML way** uses a `hibernate.cfg.xml` file to hold database connection details (URL, username, password, dialect) and lists which mapping files/classes to use. The **Annotation way** places configuration directly on the Java classes themselves (like `@Entity`, `@Id`, `@Column`), which keeps the mapping closer to the code and is the more popular approach in modern projects.

Code Example (hibernate.cfg.xml):

```

<hibernate-configuration>
<session-factory>
<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/college_db</property>
<property name="hibernate.connection.username">root</property>
<property name="hibernate.connection.password">1234</property>
<property name="hibernate.dialect">
org.hibernate.dialect.MySQLDialect</property>
<mapping class="com.example.Student"/>
</session-factory>
</hibernate-configuration>

```

6.7 Basic CRUD Operations

CRUD stands for **Create, Read, Update, Delete** — the four basic operations every application needs to perform on data. Hibernate provides simple Session methods for each of these, so you work directly with Java objects instead of writing SQL.

Operation	Hibernate Method	Purpose
Save (Create)	<code>session.save(obj)</code>	Insert a new object as a row
Retrieve (Read)	<code>session.get(Class, id)</code>	Fetch a row as a Java object
Update	<code>session.update(obj)</code>	Modify an existing row
Delete	<code>session.delete(obj)</code>	Remove a row from the table

Code Example:

```

Transaction tx = session.beginTransaction();

// CREATE
Student s = new Student();
s.setId(1);
s.setName("Indra");
session.save(s);

// READ
Student found = session.get(Student.class, 1);

// UPDATE
found.setName("Indranil");
session.update(found);

// DELETE
session.delete(found);

tx.commit();

```

6.8 Hibernate Query Language (HQL) Basics

HQL is Hibernate's own query language, and it looks very similar to SQL, but with one key difference: instead of writing queries against *table names and columns*, you write them against *Java class names and properties*. This makes queries more object-oriented and database-independent, since Hibernate translates HQL into the correct SQL for whichever database is being used underneath.

Code Example:

```

// Note: "Student" is the Java class name, "name" is its field
Query query = session.createQuery(
    "FROM Student WHERE name = :studentName");
query.setParameter("studentName", "Indra");

List<Student> results = query.list();

```

End of Syllabus — All the Best for your Semester 6 Exams, Indra!